

ИДЗ №1 по ОС.

Магомедов Абдул Омаргаджиевич. БПИ 234

Вариант ИДЗ: 1 вариант.

Условие:

Разработать программу, которая «переворачивает» заданную позициями N1–N2 часть ASCII–строки символов (числа N1, N2 вводятся как параметры).

В самом конце отчета указано, как надо запускать программы.

Программа на оценку 4:

Схема решаемой задачи:

От пользователя требуется ввести имя входного файла, имя выходного файла, N1 и N2 – индексы начала и конца по такому принципу:

«./programMark4 input.txt output.txt N1 N2»

Общая схема взаимодействия процессов через неименованные каналы:

1-й процесс $\xrightarrow{\text{pipe1}}$ 2-й процесс $\xrightarrow{\text{pipe2}}$ 3-й процесс

- **Первый процесс** — читает данные из файла и передает во второй процесс.
- **Второй процесс** — принимает строку через pipe1, обрабатывает ее (переворачивает подстроку), передает результат через pipe2 третьему процессу.
- **Третий процесс** — записывает полученную строку в выходной файл.

Работа с файлами через системные вызовы:

Все операции ввода-вывода выполняются через системные вызовы ОС Linux:

- Чтение из файла: read
- Запись в файл: write

Формат командной строки:

`./program <входной файл> <выходной файл> <N1> <N2>`

- <входной файл> — путь к входному файлу.
- <выходной файл> — путь к выходному файлу.
- <N1> — начальная позиция для переворачивания.
- <N2> — конечная позиция для переворачивания.

При некорректных параметрах программа выводит сообщение:

«Использование: `./program <входной файл> <выходной файл> <N1> <N2>`»

Так же программа не дает пользователю ввести значение N2 меньше чем N1 и больше, чем длина строки.

Размеры буферов:

Для хранения данных используется буфер размером **5000 байт**.

Работа с тестовыми файлами:

Предполагается, что весь текст полностью уместается в 5000 байт, следовательно, достаточно одного вызова **read** / **write** для передачи данных.

Тестирование на 5 тестовых файлах:

№	Входной файл	Содержимое входного файла	Параметры N1 и N2	Содержимое выходного файла
1	input4_1.txt	Hello, dlroW!	7 и 11	Hello, World!
2	input4_2.txt	esreveR	0 и 6	Reverse
3	input4_3.txt	abcdefg	3 и 6	abcgfed
4	input4_4.txt	012345	0 и 3	321045
5	input4_5.txt	kcehC	0 и 4	Check

Все файлы предоставлены в архиве.

Результаты работы программы:

При успешной обработке программа выводит: «Программа успешно завершена.»

Результат работы так же представлен в архиве.

Программа на оценку 5:

Общая схема решаемой задачи для именованных каналов:

От пользователя требуется ввести имя входного файла, имя выходного файла, а также индексы N1 и N2 (начало и конец подстроки для переворачивания) в следующем формате:

«./programMark5 input.txt output.txt N1 N2»

Схема взаимодействия процессов через именованные каналы:

1-й процесс -----> FIFO1 -----> 2-й процесс -----> FIFO2 -----> 3-й процесс

1. Первый процесс (Чтение)

- Открывает входной файл.
- Читает данные из файла в буфер.
- Передает данные через именованный канал FIFO1 второму процессу

2. Второй процесс (Обработка)

- Принимает данные из FIFO1
- Обработывает строку: переворачивает подстроку с индексами от N1 до N2.
- Передает результат обработки через именованный канал FIFO2 третьему процессу.

3. Третий процесс (Запись)

- Принимает обработанные файлы из FIFO2.
- Записывает результат в выходной файл.

Именованные каналы:

- FIFO1 – канал для передачи данных от первого процесса ко второму.
- FIFO2 – канал для передачи данных от второго процесса к третьему.

- Каналы создаются с помощью функции `mkfifo`.

Разработка консольного приложения:

Программа использует именованные каналы для передачи данных между процессами. Процессы:

1. Первый процесс читает данные из файла.
2. Второй процесс обрабатывает файл.
3. Третий процесс записывает результат в файл.

Формат командной строки:

`./program <входной файл> <выходной файл> <N1> <N2>`

- <входной файл> — путь к входному файлу.
- <выходной файл> — путь к выходному файлу.
- <N1> — начальная позиция для переворачивания.
- <N2> — конечная позиция для переворачивания.

При некорректных параметрах программа выводит сообщение:

«Использование: `./program <входной файл> <выходной файл> <N1> <N2>`»

Так же программа не дает пользователю ввести значение N2 меньше чем N1 и больше, чем длина строки.

Ввод и вывод данных:

Ввод и вывод данных осуществляются через системные вызовы `read` и `write`.

Размеры буферов:

Для хранения данных используется буфер размером **5000 байт**.

Работа с тестовыми файлами:

Предполагается, что весь текст полностью умещается в 5000 байт, следовательно, достаточно одного вызова **read** / **write** для передачи данных.

Тестирование на 5 тестовых файлах:

№	Входной файл	Содержимое входного файла	Параметры N1 и N2	Содержимое выходного файла
1	Input5_1.txt	abcolleHcba	3 и 7	abcHellocba
2	Input5_2.txt	abcdefgh	2 и 7	abhgfedc
3	Input5_3.txt	1234567890	3 и 6	1237654890
4	Input5_4.txt	tseT	0 и 3	Test
5	Input5_5.txt	xuniL Ubuntu	0 и 4	Linux Ubuntu

Все файлы предоставлены в архиве.

Результаты работы программы:

Программа успешно обрабатывает входные данные и записывает результат в выходной файл. При завершении работы выводится сообщение: "Программа выполнена успешно." Все входные и выходные файлы так же предоставлены в архиве.

Программа на оценку 6:

Общая схема решаемой задачи:

Ввод в командную строку аналогичен, как и в прошлых программах.

1-й процесс -----> pipe1 -----> 2-й процесс -----> pipe2 -----> 1-й процесс

Используются **два неименованных канала**: **pipe1** для передачи данных от родителя к потомку, и **pipe2** для обратной передачи результатов от потомка к родителю.

Разработка консольного приложения:

Родительский процесс:

1. Считывает входные данные из заданного файла (через системный вызов **read**).
2. Передаёт считанные данные через **pipe1** второму процессу.
3. После получения результатов из **pipe2**, родитель записывает результат в выходной файл (через системный вызов **write**).

Потомок (второй процесс):

1. Принимает данные из **pipe1**.
2. Выполняет реверс требуемой части строки (с позиций **N1** по **N2**).
3. Передаёт результат обратно родительскому процессу через **pipe2**.

Формат командной строки:

./programMark6 <input.txt> <output.txt> <N1> <N2>

- Если параметры заданы неверно (например, недостаточное количество аргументов), программа выводит сообщение об ошибке и завершает работу.
- Проверяется существование входного файла.
- Проверяется, что **N2** > **N1**.
- Проверяется, что **N2** не выходит за пределы фактически считанных данных.

Ввод и вывод данных:

Ввод и вывод данных осуществляются через системные вызовы **read** и **write**.

Размеры буферов:

Для хранения данных используется буфер размером **5000 байт**.

Работа с тестовыми файлами:

Предполагается, что весь текст полностью уместается в 5000 байт, следовательно, достаточно одного вызова **read** / **write** для передачи данных.

Тестирование на 5 тестовых файлах:

№	Входной файл	Содержимое входного файла	Параметры N1 и N2	Содержимое выходного файла
1	Input6_1.txt	abcolleHcba	3 и 7	abcHellocba
2	Input6_2.txt	abcdefgh	2 и 100	Ошибка
3	Input6_3.txt	1234567890	6 и 3	Ошибка
4	Input6_4.txt	tseT	0 и 3	Test
5	Input6_5.txt	xuniL Ubuntu	0 и 4	Linux Ubuntu

Все файлы предоставлены в архиве.

Ошибка 2-го теста: значение N2 не должно превышать длину строки.

Ошибка 3-го теста: значения N2 должно быть больше N1.

Результаты работы программы:

Программа успешно обрабатывает входные данные и записывает результат в выходной файл. При завершении работы выводится сообщение: "Программа выполнена успешно." Все входные и выходные файлы так же предоставлены в архиве.

Программа на оценку 7:

Общая схема решаемой задачи:

Ввод в командную строку аналогичен, как и в прошлых программах.

Процесс 1 (родитель) считывает текстовые данные из входного файла и передает их по именованному каналу FIFO1.

Процесс 2 (дочерний) читает данные из FIFO1, обрабатывает (переворачивает подстроку с позиции **N1** до **N2**) и отправляет результат обратно в Процесс 1 через FIFO2.

Процесс 1 получает из FIFO2 обработанную строку и записывает её в выходной файл.

В программе создаются именованные каналы с помощью `mkfifo("fifo1", 0666)` и `mkfifo("fifo2", 0666)`. По завершении работы каналы удаляются вызовами `unlink("fifo1")` и `unlink("fifo2")`.

Разработка консольного приложения:

Как уже было указано в схеме решаемой задачи, родительский процесс считывает данные из входного файла, затем передает их по именованному каналу FIFO1, дочерний процесс же читает файлы из канала, и отправляет результат в родительский процесс через FIFO2. Далее родительский процесс получает обработанную строку и записывает ее в выходной файл.

Формат командной строки:

`./programMark6 <input.txt> <output.txt> <N1> <N2>`

- Если параметры заданы неверно (например, недостаточное количество аргументов), программа выводит сообщение об ошибке и завершает работу.
- Проверяется существование входного файла.
- Проверяется, что **N2** > **N1**.
- Проверяется, что **N2** не выходит за пределы фактически считанных данных.

Ввод и вывод данных:

Ввод и вывод данных осуществляются через системные вызовы `read` и `write`.

Размеры буферов:

Для хранения данных используется буфер размером **5000 байт**.

Работа с тестовыми файлами:

Предполагается, что весь текст полностью помещается в 5000 байт, следовательно, достаточно одного вызова **read** / **write** для передачи данных.

Тестирование на 5 тестовых файлах:

№	Входной файл	Содержимое входного файла	Параметры N1 и N2	Содержимое выходного файла
1	Input7_1.txt	Hello, dlroW!	7 и 11	Hello, World!
2	Input7_2.txt	esreveR	0 и 6	Reverse
3	Input7_3.txt	abcdefg	3 и 6	abcgfed
4	Input7_4.txt	012345	0 и 3	321045
5	Input7_5.txt	kcehC	0 и 4	Check

Все файлы предоставлены в архиве.

Результаты работы программы:

Программа успешно обрабатывает входные данные и записывает результат в выходной файл. При завершении работы выводится сообщение: "Программа выполнена успешно." Все входные и выходные файлы так же предоставлены в архиве.

Программа на оценку 8:

Общая схема решаемой задачи:

У нас есть **два независимых процесса** (две разные программы):

1. **writer8** (первый процесс)

- Считывает данные из заданного входного файла.
- Передаёт эти данные через именованный канал **FIFO1** второму процессу.

2. **processor8** (второй процесс)

- Принимает данные через **FIFO1**.
- Обработывает строку (переворачивает часть [N1..N2]).
- Передаёт обработанные данные обратно через **FIFO2**.

После получения результата по каналу **FIFO2**, процесс **writer8** записывает итоговые данные в выходной файл.

Разработка консольного приложения:

- Программа **writer8**:
 - Аргументы: **./writer8 <входной файл> <выходной файл> <N1> <N2>**.
 - Создаёт (при необходимости) именованные каналы **fifo1** и **fifo2**.
 - Считывает данные из входного файла и пересылает их в **fifo1**.
 - Затем читает результат из **fifo2** и сохраняет его в выходной файл.
- Программа **processor8**:
 - Аргументы: **./processor8 <N1> <N2>**.
 - Считывает данные из **fifo1**, обрабатывает (переворот подстроки) и отправляет результат в **fifo2**.

Обе программы используют **системные вызовы open, read, write** для работы с файлом и FIFO.

Формат командной строки:

Запуск первой программы (**writer8**):

- **./writer8 <input8_1.txt> <output8_1.txt> <N1> <N2>**

При некорректном числе аргументов, отрицательных значениях или **N2 < N1**, выводится соответствующее сообщение об ошибке.

Запуск второй программы (**processor8**):

- ./processor8

Параметры **N1**, **N2** извлекаются из принятого потока данных.

Ввод и вывод данных через системные вызовы:

Используются системные вызовы open, read, write, close для работы с файлами и FIFO.

Размеры буферов:

Для хранения данных используется буфер размером **5000 байт**.

Работа с тестовыми файлами:

Предполагается, что весь текст полностью уместается в 5000 байт, следовательно, достаточно одного вызова **read** / **write** для передачи данных.

Тестирование на 5 тестовых файлах:

№	Входной файл	Содержимое входного файла	Параметры N1 и N2	Содержимое выходного файла
1	Input5_1.txt	abcolleHcba	3 и 7	abcHellocba
2	Input5_2.txt	abcdefgh	2 и 7	abhgfedc
3	Input5_3.txt	1234567890	3 и 6	1237654890
4	Input5_4.txt	tseT	0 и 3	Test
5	Input5_5.txt	xuniL Ubuntu	0 и 4	Linux Ubuntu

Все файлы предоставлены в архиве.

Результаты работы программы:

Программа успешно обрабатывает входные данные и записывает результат в выходной файл. При завершении работы выводится сообщение:

"Программа выполнена успешно." Все входные и выходные файлы так же предоставлены в архиве.

Программа на оценку 9:

Общая схема решаемой задачи:

Существует два независимых процесса из разных программ: **writer9** и **processor9**.

writer9 открывает входной файл, читает его порциями по 128 байт и передаёт через именованный канал *fifo1* процессу **processor9**.

После завершения передачи данных **writer9** закрывает *fifo1* и открывает *fifo2* для чтения результата.

processor9 читает приходящие порции через *fifo1*, аккумулирует их во внутреннем буфере произвольной длины, а затем выполняет «переворот» (reversing) части строки с индексами N1–N2.

Результат передаётся обратно по *fifo2* тоже порциями (не более 128 байт за раз), а **writer9** считывает эти порции и записывает их в выходной файл.

Разработка консольного приложения:

Приложение состоит из двух программ: **writer9** (чтение файла, передача данных, вывод результата) и **processor9** (обработка строки).

Взаимодействие:

- *fifo1* используется для передачи исходных данных от **writer9** к **processor9**;
- *fifo2* используется для передачи готового результата обратно.

Формат командной строки:

Запуск первой программы (**writer9**):

- `./writer9 <input9_1.txt> <output9_1.txt> <N1> <N2>`

При некорректном числе аргументов, отрицательных значениях или $N2 < N1$, выводится соответствующее сообщение об ошибке.

Запуск второй программы (**processor9**):

- ./processor9

Параметры **N1**, **N2** извлекаются из принятого потока данных.

Ввод и вывод данных через системные вызовы:

Используются системные вызовы open, read, write, close для работы с файлами и FIFO.

Размеры буферов:

Пересылка между процессами осуществляется блоками размера:

BUF_SIZE = 128.

Внутренний (промежуточный) буфер внутри **processor9** может быть больше 128, так как требуется собрать все данные и выполнить единый «переворот» подстроки.

Таким образом, канал получает и отправляет порции ровно по 128 байт, удовлетворяя условию об ограниченном размере буфера пересылки.

Работа с тестовыми файлами:

В **writer9** реализован цикл построчного чтения входного файла, пока не будет достигнут конец. Все данные передаются через *fifo1* кусками по 128 байт.

Это позволяет обрабатывать файлы практически любого размера.

Во втором процессе (**processor9**) данные аккумулируются в динамически расширяемом буфере (так как надо иметь все данные сразу для выполнения «переворота» по индексам $N1-N2$).

Тестирование на 5 тестовых файлах:

№	Входной файл	Содержимое входного файла	Параметры N1 и N2	Содержимое выходного файла
---	--------------	---------------------------	-------------------	----------------------------

1	Input9_1.txt	Hello, dlroW!	7 и 11	Hello, World!
2	Input9_2.txt	esreveR	0 и 6	Reverse
3	Input9_3.txt	abcdefg	3 и 6	abcgfed
4	Input9_4.txt	012345	0 и 3	321045
5	Input9_5.txt	kcehC	0 и 4	Check

Все файлы предоставлены в архиве.

Результаты работы программы:

Программа успешно обрабатывает входные данные и записывает результат в выходной файл. При завершении работы выводится сообщение: "Программа выполнена успешно." Все входные и выходные файлы так же предоставлены в архиве.

Программа на оценку 10:

Общая схема решаемой задачи:

Есть два независимых процесса — **writer10** и **processor10**. Первый читает файл кусками (макс. 128 байт) и передаёт их второму через очередь сообщений. Второй процесс накапливает все данные в памяти, выполняет переворот подстроки по индексам N1–N2, затем передаёт результат обратно в **writer10** по другой очереди сообщений.

Разработка консольного приложения:

Программы **writer10** и **processor10** взаимодействуют друг с другом по двум системным очередям с ключами, '0x1234' и '0x2345', с помощью системных вызовов `msgget`, `msgsnd`, `msgrcv`. Данные кусками до 128 байт.

Формат командной строки:

Запуск первой программы (**writer10**):

- `./writer10 <input10_1.txt> <output10_1.txt> <N1> <N2>`

При некорректном числе аргументов, отрицательных значениях или **N2 < N1**, выводится соответствующее сообщение об ошибке.

Запуск второй программы (**processor10**):

- `./processor10`

Параметры **N1**, **N2** извлекаются из принятого потока данных.

Ввод и вывод данных через системные вызовы:

Используются системные вызовы `open`, `read`, `write`, `close` для работы с файлами.

Размеры буферов:

Обработка (переворот подстроки) при необходимости выполняется в более крупном буфере внутри **processor10**, так как по частям разворачивать заданную подстроку бывает неудобно или невозможно, если надо учитывать точные индексы.

Тестирование на 5 тестовых файлах:

№	Входной файл	Содержимое входного файла	Параметры N1 и N2	Содержимое выходного файла
1	Input10_1.txt	Hello, dlroW!	7 и 11	Hello, World!
2	Input10_2.txt	esreveR	0 и 6	Reverse
3	Input10_3.txt	abcdefg	3 и 6	abcgfed
4	Input10_4.txt	012345	0 и 3	321045
5	Input10_5.txt	kcehC	0 и 4	Check

Все файлы предоставлены в архиве.

Результаты работы программы:

Программа успешно обрабатывает входные данные и записывает результат в выходной файл. При завершении работы выводится сообщение: "Программа выполнена успешно." Все входные и выходные файлы так же предоставлены в архиве.

Как запускать программы:

С программами на оценку 4-7 все понятно, так как там всего один С файл, ну и в критериях на каждую описан формат необходимой строки. А в программах на оценку 8-10 нужно делать по такой очередности:

Сперва в одной консоли запускаем `writer.c`, вводим все параметры правильно и убеждаемся, что `writer` находится в режиме ожидания, после этого открываем новую консоль и просто запускаем `processor`.

Теперь как считаются переворот подстроки по индексам:

К примеру, у нас есть строка «0123456789». Если мы условно хотим перевернуть подстроку «567» то нам нужно ввести значения N1 и N2 – 5 и 7. То есть мы переворачиваем строку от индекса N1 до индекса N2 включительно.